

[Vai al contenuto](#)

DISPENSA DELLA LEZIONE

React Context

Condividere un contatore tra componenti senza passare props inutili.



CAPITOLO 1

Il problema: il prop drilling

NOTA

Il **prop drilling** avviene quando una prop attraversa componenti che non la usano, solo per raggiungere quello che ne ha bisogno.

In React i dati viaggiano normalmente dall'alto verso il basso: un componente padre passa una prop a un componente figlio. È un meccanismo semplice e preferibile quando i componenti sono vicini nell'albero.

Immagina però un contatore: **Total** deve leggere il valore, mentre **Click** deve aggiornarlo. Senza Context, lo stato dovrebbe vivere in **App** ed essere passato come prop ai componenti interessati.

```
function App() {  
  const [counter, setCounter] = useState(0);  
  
  return (  
    <>  
      <Total counter={counter} />  
      <Click setCounter={setCounter} />  
    </>  
  );  
}
```

Con due componenti questa soluzione è assolutamente valida. Il problema emerge quando, tra **App** e il componente che usa il dato, ci sono altri componenti. Questi ricevono prop che non gli servono e le inoltrano soltanto: il codice diventa più rumoroso e ogni modifica alla struttura dell'app richiede di aggiornare più file.

CAPITOLO 2

Creare il Context

NOTA

Un Context è un canale condiviso: il Provider inserisce un valore e i componenti al suo interno possono consumarlo.

Context non è uno stato alternativo a `useState`: è un modo per rendere disponibile uno stato a più componenti. Nel nostro esempio lo stato continua a essere creato con `useState`; Context distribuisce il valore e la funzione che lo modifica.

Creiamo il file `src/contexts/CounterContext.jsx`. `createContext()` crea il contesto che conterrà il contatore.

```
import { createContext, useContext, useState } from "react";  
  
const CounterContext = createContext();
```

Il valore passato a `createContext()` è il valore di riserva, usato solo se un componente legge il Context senza trovarsi dentro il Provider. Qui non ne specifichiamo uno perché i nostri componenti devono essere usati esclusivamente dentro `CounterProvider`.

Il contesto non viene esportato direttamente: esporteremo invece il Provider e un hook dedicato, così i componenti avranno un modo semplice e coerente per usarlo.

CAPITOLO 3

CounterProvider: stato e condivisione

NOTA

Il Provider deve avvolgere i componenti che devono leggere o aggiornare il valore condiviso.

Un Provider è un normale componente React. Riceve automaticamente la prop speciale `children`, cioè tutto ciò che scriviamo tra il tag di apertura e quello di chiusura. In questo caso, `children` saranno `Total` e `Click`.

`CounterProvider` conserva lo stato con `useState(0)` e passa sia `counter` sia `setCounter` nel valore del Provider. Mettiamo i due valori in un oggetto perché i componenti possano estrarre solo ciò che serve loro.

```
const CounterProvider = ({ children }) => {  
  const [counter, setCounter] = useState(0);  
  
  return (  
    <CounterContext.Provider value={{ counter, setCounter }}>  
      {children}  
    </CounterContext.Provider>  
  );  
};
```

Ogni volta che `setCounter` cambia lo stato, il Provider riceve un nuovo valore. React aggiorna quindi tutti i componenti che stanno consumando quel Context: non occorre inviare manualmente nuove props.

CAPITOLO 4

Un hook per consumare il Context

NOTA

Un hook personalizzato nasconde il dettaglio di `useContext(CounterContext)` dietro un nome più chiaro: `useCount`.

`useContext` riceve il Context e restituisce il valore del Provider più vicino nell'albero. Per convenzione, gli hook personalizzati iniziano con `use`; questo comunica a React e a chi legge il codice che la funzione usa un hook.

Raccogliere `useContext(CounterContext)` dentro `useCount` ha un vantaggio pratico: se in futuro cambiamo il modo di gestire il contatore, i componenti `Total` e `Click` continuano a usare la stessa interfaccia.

Completiamo ed esportiamo il file `src/contexts/CounterContext.jsx`:

```
import { createContext, useContext, useState } from "react";

const CounterContext = createContext();

const CounterProvider = ({ children }) => {
  const [counter, setCounter] = useState(0);

  return (
    <CounterContext.Provider value={{ counter, setCounter }}>
      {children}
    </CounterContext.Provider>
  );
};

function useCount() {
  return useContext(CounterContext);
}

export { CounterProvider, useCount };
```

CAPITOLO 5

Usare il contatore nei componenti

In `App`, `CounterProvider` avvolge `Total` e `Click`. La posizione è importante: solo i discendenti del Provider possono usare `useCount`. I due componenti non ricevono props.

File `src/App.jsx`:

```
import Click from "../components/Click"
import Total from "../components/Total"
import { CounterProvider } from "../contexts/CounterContext"

export default function App() {
  return (
    <CounterProvider>
      <Total />
      <Click />
    </CounterProvider>
  );
}
```

`Total` legge soltanto `counter`. Destrutturare solo il valore necessario rende evidente la responsabilità del componente: visualizzare il totale, senza modificarlo.

File `src/components/Total.jsx`:

```
import { useCount } from "../contexts/CounterContext";

export default function Total() {
  const { counter } = useCount();

  return <h1>{counter}</h1>;
}
```

`Click`, al contrario, usa soltanto `setCounter`. Nella funzione di aggiornamento scriviamo `current => current + 1`: React fornisce sempre il valore più recente dello stato. Questa forma è preferibile quando il nuovo valore dipende da quello precedente.

File `src/components/Click.jsx`:

```
import { useCount } from "../contexts/CounterContext";

export default function Click() {
  const { setCounter } = useCount();

  return (
    <button onClick={() => setCounter(current => current + 1)}>
      Cliccami
    </button>
  );
}
```

CAPITOLO 6

Quando usare Context

NOTA

Usa Context per dati condivisi tra più componenti; usa props e stato locale quando il dato ha pochi destinatari.

Context è utile per dati come utente autenticato, lingua, tema o stato condiviso in vari punti dell'app. In questo esempio evita di passare il contatore e la sua funzione di aggiornamento come props.

Non è necessario creare un Context per ogni dato. Se uno stato è usato da un solo componente, `useState` locale resta la soluzione più semplice. Anche una prop passata a un figlio diretto è spesso più leggibile di un Context.

È inoltre una buona pratica creare Context piccoli e con una responsabilità chiara: un `CounterContext` per il contatore, un eventuale `ThemeContext` per il tema e così via. Un unico Context enorme con dati non collegati rende più difficile capire da dove arrivano le informazioni.

Ricorda infine che quando cambia il valore di un Provider, React rivaluta i componenti che consumano quel Context. Per valori che cambiano molto spesso o per applicazioni molto grandi possono servire altre tecniche di ottimizzazione; nella maggior parte delle interfacce, però, Context è una soluzione chiara e sufficiente.